

PARAMETER TAMPERING

INTRODUCTION

In modern web applications, data is often exchanged between clients and servers through parameters in URLs, forms, cookies, and headers. While these parameters are critical for application functionality, they also present a potential attack surface. **Parameter Tampering** and **HTTP Parameter Pollution (HPP)** are two common attack vectors where attackers manipulate or inject parameters to exploit application logic. These vulnerabilities can lead to authentication bypass, unauthorized access, financial fraud, and other severe impacts if not properly mitigated.

1. What is Parameter Tampering?

Parameter Tampering is a **web application attack** where an attacker **manipulates parameters exchanged between client and server** (e.g., URL parameters, form fields, cookies, or headers).

- Goal: To bypass security controls, gain unauthorized access, or manipulate data.
- Example: Changing price=100 to price=1 in an e-commerce checkout request.

2. How HTTP Parameter Pollution Works?

HTTP Parameter Pollution (HPP) is a technique where attackers **inject duplicate or unexpected parameters** into an HTTP request to bypass filters, validation, or exploit vulnerabilities.

Example:

`https://example.com/login?user=admin&user=attacker`

- If the application processes the first value → attacker can override it.
- If the app concatenates values → unexpected behavior may occur.

Steps:

1. Attacker injects extra parameters in GET/POST requests.
2. Server interprets them differently depending on coding logic.
3. This may bypass authentication, filters, or business rules.

3. Total Vulnerabilities from HTTP Parameter Pollution Attack

HPP can introduce multiple vulnerabilities such as:

1. **Authentication Bypass** – modifying login parameters.
2. **Access Control Bypass** – altering role IDs, privileges.
3. **Data Manipulation** – tampering with prices, quantities, or sensitive data.
4. **Information Disclosure** – extracting hidden fields or debug info.
5. **Cross-Site Scripting (XSS) or SQL Injection** – injecting payloads via parameters.

4. Impact of Parameter Tampering Attack

- **Financial Fraud** – Changing payment amounts or discounts.
- **Unauthorized Access** – Escalating privileges by modifying user roles.
- **Data Theft** – Extracting sensitive information.
- **Bypassing Security Controls** – Skipping input validation or filters.
- **Reputation Damage** – If exploited on a public platform, it can cause trust loss.

5) Mitigations

To prevent Parameter Tampering & HPP:

1. **Server-side Validation** – Never trust client-side input. Validate all parameters on the server.
2. **Strong Authentication & Authorization** – Ensure parameter changes don't give unauthorized access.
3. **Input Filtering & Whitelisting** – Allow only expected parameter values.
4. **Use Tokens (CSRF protection)** – To prevent replay or forged requests.
5. **Web Application Firewall (WAF)** – Detect and block abnormal parameter injection.
6. **Avoid depending solely on hidden fields/cookies** – since attackers can tamper with them easily.

CONCLUSION

Parameter Tampering and HTTP Parameter Pollution are dangerous yet often overlooked vulnerabilities in web applications. By altering parameters, attackers can manipulate application logic, perform unauthorized actions, or gain access to sensitive data. The impacts can range from minor business losses to large-scale breaches. However, with proper **input validation**, **strong authentication**, and **layered security measures**, organizations can effectively mitigate these risks. Building applications with **secure coding practices** and regular security testing is essential to stay protected against such attacks.